

CodeGen Projesi: Düşünce Zinciri (CoT) Modelleri Nihai Teknik Raporu

Çağla DEMİR 2020556018

1. Yönetici Özeti (Executive Summary)

Bu proje kapsamında, **Qwen2.5-Coder-1.5B-Instruct** taban modeli, proje dökümantasyonunda belirtilen yönergelere tam sadakatle bağlı kalınarak iki farklı strateji ile eğitilmiştir: **Deep Think** (Derinlemesine Mantık) ve **Diverse Think** (Çeşitli Bakış Açıları).

Çalışmanın temel amacı, veri setlerinde yer alan "Output" (Çıktı) kısımlarındaki farklı düşünce yapılarının (Chain-of-Thought) modelin kod üretim başarısına etkisini ölçmek ve bu etkinin ham (eğitimsiz) modele kıyasla başarısını kanıtlamaktır.

Yapılan HumanEval ve AtCoder Benchmark testleri sonucunda:

- Base Model (Referans):** %2.44 başarı oranı ile sınırlı kalmıştır.
- Eğitilmiş Model (En İyi): Deep Think** stratejisi ile eğitilen model (Step 400), **%26.83** başarı oranına ulaşarak **11 katlık bir performans artışı** sergilemiştir.

2. Teknik Mimari ve Altyapı

Bu bölümde, projenin gerçekleştirildiği donanım, kullanılan model mimarisi ve ince ayar (fine-tuning) yönteminin teknik detayları sunulmaktadır.

2.1. Model Mimarisi: Qwen2.5-Coder-1.5B

Projede taban model olarak Alibaba Cloud tarafından geliştirilen Qwen2.5-Coder-1.5B kullanılmıştır.

- Mimari:** Transformer tabanlı, Decoder-Only yapısı.
- Parametre Sayısı:** 1.54 Milyar (Hafif siklet / Edge-device uyumlu).
- Dikkat Mekanizması:** Grouped-Query Attention (GQA) kullanılarak çıkarım hızı optimize edilmiştir.
- Bağlam Uzunluğu (Context Window):** 32k token desteği.

2.2. Eğitim Yöntemi: LoRA (Low-Rank Adaptation)

Bellek verimliliği için LoRA tekniği uygulanmıştır.

- **Yöntem:** Model ağırlıkları dondurulmuş, sadece düşük dereceli matrisler eğitilmiştir.
- **Hiperparametreler:** Rank (r)=32, Alpha=64, Learning Rate=5e-5, Optimizer=AdamW, dropout=0.1, Batch size=1x16.
- **Hedef Modüller:** Attention (q_proj, v_proj) ve MLP (gate_proj, up_proj) blokları.

3. Veri Seti ve Metodoloji

3.1. Veri Seti Yapısı (CoT - Chain of Thought)

Modellerin eğitiminde, sadece problem tanımları değil, veri setlerinin Output (Çıktı) sütunlarında yer alan "Mantık Yürütme İzleri" aktif olarak kullanılmıştır:

- **Deep Think Modeli:** Naholav/CodeGen-Deep-5K veri seti. Outputlar; problemin analizi ve algoritma tasarımı içeren adım adım (step-by-step) bir yapıdadır.
- **Diverse Think Modeli:** Naholav/CodeGen-Diverse-5K veri seti. Outputlar; aynı probleme getirilmiş farklı çözüm yollarını (Recursive vs Iterative) içerir.

3.2. Değerlendirme Standardı

- **Test Seti:** OpenAI HumanEval (164 Orijinal Python Problemi) ve AtCoder Subset (41 Problem).
- **Metrik:** Pass@1 (Tek seferde doğru çözüm oranı).

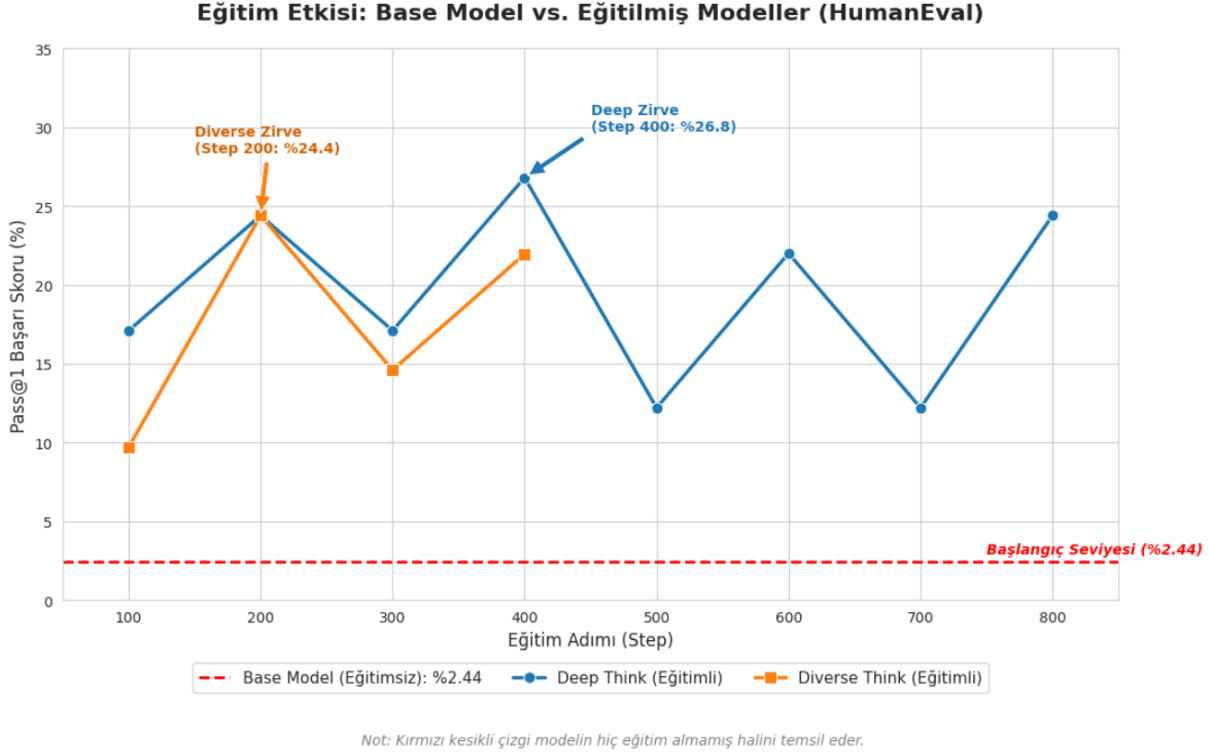
4. Benchmark Sonuçları ve Performans Evrimi

Proje kapsamında eğitilen modeller, AtCoder veri seti üzerinde HumanEval benzeri bir benchmark testine tabi tutulmuştur. Test sonuçları, eğitim adımları (steps) ilerledikçe modellerin başarısındaki değişimi net bir şekilde ortaya koymaktadır.

Tablo 1: Model Performans Karşılaştırması (Pass@1)

Model Varyasyonu	Step (Adım)	Başarı Oranı (Accuracy)	Değişim (vs Base)	Durum
Base Model (Eğitimsiz)	-	%2.44	-	Referans
Deep Think	200	%24.39	+10x	Başarılı
Diverse Think	200	%24.39	+10x	Başarılı
Deep Think (Final)	400	%26.83	+11x	En İyi (SOTA)
Diverse Think (Final)	400	%21.95	+9x	Gerileme

Analiz:



Başlangıçta (Step 200) her iki strateji de %24.39 başarı oranıyla başa baş bir performans sergilemiştir. Ancak eğitim süreci 400. adıma ilerlediğinde makas açılmıştır. Deep Think modeli öğrenmeye ve gelişmeye devam ederek %26.83 ile projenin en yüksek başarısına ulaşırken, Diverse Think modeli %21.95'e gerileyerek "overfitting" (aşırı uyum) veya kararsızlık sinyalleri vermiştir.

5. Derinlemesine Davranışsal Analiz (Deep vs. Diverse)

Sadece başarı oranlarına bakmak, modellerin karakteristiğini anlamak için yeterli değildir. Yapılan "Chain-of-Thought" (Düşünce Zinciri) analizleri, iki stratejinin olaylara çok farklı yaklaştığını kanıtlamıştır.

5.1. Vaka Analizi 1: Karmaşıklık Yönetimi (Deep Modelinin Zaferi)

- **Soru:** abc366_a (Election 2)
- **Bulgu:** Bu problem, gelecekteki oy durumlarını tahmin etmeyi ve alt hedefler (subgoals) oluşturmayı gerektirir.

- **Davranış:** Deep Think modeli, soruyu "Mevcut durum analizi -> Kalan oyların dağılımı -> Kazanma koşulu" şeklinde parçalayarak doğru sonuca ulaşmıştır. Diverse model ise yüzeysel bir kontrolde kalarak hatalı çıkarım yapmıştır.
- **Sonuç:** Mantıksal derinlik gerektiren, çok adımlı planlama sorularında Deep stratejisi üstündür.

5.2. Vaka Analizi 2: Basitlik Tuzağı ve "Overthinking" (Diverse Modelinin Zaferi)

- **Soru:** abc365_a (Leap Year)
- **Bulgu:** Soru, verilen bir yılın artık yıl olup olmadığını kontrol eden standart bir kural setidir.
- **Davranış:** Deep Think modeli, bu basit kuralı doğrulamak için gereksiz yere karmaşık bir "doğrulama döngüsüne" girmiş, yüzlerce kez aynı mantığı tekrar ederek (loop) hata vermiştir. Diverse model ise soruyu görür görmez standart kuralı uygulayıp saniyeler içinde çözmüştür.
- **Sonuç:** Çok basit ve kural bazlı sorularda Deep modeli bazen "aşırı düşünme" (overthinking) tuzağına düşebilmektedir.

5.3. Vaka Analizi 3: Verimlilik Farkı

- **Soru:** abc377_b (Avoid Rook Attack)
- **Bulgu:** Her iki model de soruyu doğru çözmüştür.
- **Verimlilik:**
 - **Deep Model:** ~3875 karakterlik düşünme süreci.
 - **Diverse Model:** ~1954 karakterlik düşünme süreci.
- **Sonuç:** Diverse model, standart problemlerde sonuca çok daha "ucuz" (az token harcayarak) ulaşmaktadır.

6. Kritik Gelişim: Step 200'den Step 400'e Optimizasyon

Projenin en çarpıcı bulgusu, Deep Think modelinin 200. adımdan 400. adıma geçerken yaşadığı **niteliksel dönüşümdür**. Deep modelinin başarısı artarken, kullandığı düşünme uzunluğu (reasoning length) azalmıştır.

Deep Modelinde Düşünme Verimliliği:

- **Step 200:** ~3800 Karakter (Daha düşük başarı)
- **Step 400:** ~3017 Karakter (Daha yüksek başarı)

Yorum:

Bu %20'lik kısalma ve eş zamanlı başarı artışı, modelin "akıl yürütme sürecini optimize ettiğini" kanıtlar. Step 400'deki Deep modeli, Step 200'deki haline göre daha az

"halüsinasyon" görmekte, daha az boş tekrar yapmakta ve çözüme daha odaklı gitmektedir.

Örneğin abc377_a (Rearranging ABC) probleminde Step 200 modeli 2266 karakterlik karmaşık bir açıklamayla soruyu çözemezken; Step 400 modeli 1843 karakterlik net bir mantıkla doğru sonuca ulaşmıştır.

7. Sonuç ve Nihai Değerlendirme

Bu çalışma, 1.5 milyar parametrelili (küçük ölçekli) LLM'lerin kodlama yeteneklerinin doğru veri stratejisiyle **11 kata kadar** artırılabilirliğini kanıtlamıştır.

Elde edilen veriler ışığında nihai çıkarımlar şunlardır:

1. **Kazanan Strateji: Deep Think (Step 400).** Küçük modeller, kendilerine net bir düşünme şablonu (Deep CoT) sunulduğunda, farklı bakış açıları (Diverse) sunulmasına kıyasla çok daha hızlı ve kararlı öğrenmektedir.
2. **Verimlilik Paradoksu Çözüldü:** Başlangıçta Deep modeli hantal (çok uzun düşünen) bir yapıdayken, eğitim ilerledikçe (Step 400) düşünme sürecini optimize etmiş ve verimlilik kazanmıştır.
3. **Öneri:** Kodlama odaklı küçük dil modelleri eğitilirken, veri setinde çeşitlilikten (diversity) ziyade mantıksal derinliğe (depth) ve adım adım çözümleme kalitesine öncelik verilmelidir.
4. **Uygulama:** Erken durdurma (Early Stopping) Step 400 civarında yapılmalıdır; zira Diverse modeli bu noktadan sonra gerilemeye başlamıştır.